Lab 54: 55% first-contact resolution: calibrate with criteria and examples

Topic 5: Context Management & Reliability · Task 5.2: Design effective escalation and ambiguity resolution patterns

Runs on Simulator 1: Customer Support Resolution Agent · Run in class

Source: Official exam guide, sample question 3

The question

Your agent achieves 55% first-contact resolution, well below the 80% target. Logs show it escalates straightforward cases (standard damage replacements with photo evidence) while attempting to autonomously handle complex situations requiring policy exceptions. What's the most effective way to improve escalation calibration?

A. Add explicit escalation criteria to your system prompt with few-shot examples demonstrating when to escalate versus resolve autonomously.

B. Have the agent self-report a confidence score (1-10) before each response and automatically route requests to humans when confidence falls below a threshold.

C. Deploy a separate classifier model trained on historical tickets to predict which requests need escalation before the main agent begins processing.

D. Implement sentiment analysis to detect customer frustration levels and automatically escalate when negative sentiment exceeds a threshold.

Simulate the solution

```
# In the browser: open Simulator 1, then Lab 54
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 54

# In a terminal, from simulators/simulator-1-customer-support/
uv run support_agent.py --case CS-05 --case CS-08 --case CS-09 --case CS-10
uv run evaluate.py
```

What you should see

Human request and price match escalate. Two names: it asks. Angry but routine: resolved.

Open `simulators/simulator-1-customer-support/evidence/` and find it yourself.

The answer

A. Explicit escalation criteria with few-shot examples.

Adding explicit escalation criteria with few-shot examples directly addresses the root cause: unclear decision boundaries. This is the proportionate first response before adding infrastructure. Option B fails because LLM self-reported confidence is poorly calibrated—the agent is already incorrectly confident on hard cases. Option C is over-engineered, requiring labeled data and ML infrastructure when prompt optimization hasn't been tried. Option D solves a different problem entirely; sentiment doesn't correlate with case complexity, which is the actual issue.

Now change one thing

Copy a file in cases/ to a new CS id, change the customer's message and expected disposition, rerun with --case, then score it with evaluate.py.